

Binary Classification of Breast Cancer Data

1. Introduction

This project is designed as a step-by-step introduction to binary classification using real data. If you have never worked with machine learning before, don't worry—this report will walk you through everything from the dataset to the final model evaluation.

What is binary classification?

Binary classification means predicting one of two possible outcomes.

Examples include:

- “cancer” vs “no cancer”
- “spam email” vs “not spam”
- “fraudulent” vs “legitimate”

In this project, we will classify tumors as either malignant (cancerous) or benign (non-cancerous)

To do this, we use three different types of algorithms:

- Random Forest – a tree-based ensemble method
- MLP (Multilayer Perceptron) – a simple form of Deep Learning
- SVM (Support Vector Machine) – a classic and powerful ML algorithm

All models were evaluated using the same method and same metrics, so even a beginner can compare them fairly.

2. Dataset

Name: Breast Cancer Wisconsin (Diagnostic)

Source: UCI Machine Learning Repository

Link: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

The dataset contains 569 samples with 30 numerical features. The target variable is binary and the labels were remapped to define the positive class (1) as malignant. Standardization was also applied and no missing values required cleaning.

3. Algorithms Overview

In this section, we explain each algorithm in a way that someone brand new can understand.

1. Required Algorithm: Random Forest

Think of a Random Forest like asking hundreds of decision trees to vote on the answer. Each tree learns something slightly different. When combined, the forest becomes much stronger than any single tree. It's great for beginners because it "just works" on many datasets. The reason why it works well here is because the dataset has many features, and trees are very good at handling high-dimensional, tabular data.

2. Deep Learning Model: MLP (Multilayer Perceptron)

An MLP is the simplest form of a neural network. You can think of it as a stack of layers where each layer transforms the data and the network learns patterns automatically.

Our network had:

Input layer: 30 features

Hidden layers: (64 neurons → 32 neurons)

Output layer: 1 prediction (malignant or benign)

This is useful because MLPs can detect subtle patterns that simpler models might miss.

3. Classic ML Algorithm: SVM (Support Vector Machine)

SVM tries to find the best possible dividing line (or curved boundary) between the two classes. Think of placing a wall between red dots (malignant) and blue dots (benign). SVM tries to position the wall so far away from both groups that it separates them cleanly. Using the RBF kernel, SVM can even learn curved boundaries. The reason why SVM is powerful here is because the data is high-dimensional and after scaling, SVM performs extremely well. It is often one of the top algorithms for small/medium tabular datasets

4. Implementation

This section explains exactly how to run the machine-learning program, even if you are new to Python. You can follow these steps using your system terminal.

Step 0 — Press Win + R, type cmd, then press Enter or Right-click inside the project folder and click open in terminal

Step 1 — Navigate to the project folder after extracting it to your desired location.

```
cd path/to/walters_taymar_finalproject
```

Step 2 — Activate your virtual environment

Windows:

```
venv\Scripts\activate
```

Mac / Linux:

```
source venv/bin/activate
```

Step 3 — Run the script

```
python walters_taymar_finalproject.py
```

A new folder called "results_dm_final" will be generated in the project folder where the tables will be saved automatically. While the program is running, you will see four different graphs at a time and each graph will pop up in a separate window. Before you exit each window PLEASE be sure to save each image in the project folder with the corresponding names in the order they appear in:

- ROC_LogisticRegression.png
- ROC_SVM_RBF.png
- ROC_RandomForest.png
- ROC_MLP.png

After running the program, your project folder will now contain the following files:

- LogisticRegression_10fold_metrics.csv
- SVM_RBF_10fold_metrics.csv
- RandomForest_10fold_metrics.csv
- MLP_10fold_metrics.csv
- Models_Summary_Averages.csv
- ROC_LogisticRegression.png
- ROC_SVM_RBF.png
- ROC_RandomForest.png
- ROC_MLP.png

5. Evaluation Setup

We used 10-fold Stratified Cross-Validation, which means:

1. The dataset is split into 10 equal parts.
2. Nine parts train the model.
3. One part tests it.
4. Repeat until each part has been used once.

This gives a fair, reliable measure of performance.

For each fold, we computed:

- TP, TN, FP, FN
- Accuracy
- Precision
- Recall (Sensitivity)
- FPR
- FNR
- Specificity (TNR)
- Balanced Accuracy
- F1 Score
- TSS
- HSS
- Brier Score (BS)
- Brier Skill Score (BSS)
- ROC curve
- AUC

These give both classification performance and probability calibration.

6. Results

Each algorithm produced per-fold metrics stored in CSV files. ROC curves were generated per algorithm. Summary tables compare performance across models.

7. Discussion

The SVM model achieved the strongest discrimination (highest AUC, TSS, and Balanced Accuracy). Random Forest showcased stability and strong recall for malignant cases. MLP performed competitively but was more sensitive to hyperparameters. Calibration varied, with MLP and logistic-style models generally outperforming Random Forest in Brier metrics.

8. Conclusion

SVM was the best discriminator, Random Forest was the most robust, and MLP offered a powerful nonlinear modeling approach. Proper evaluation methodology and model selection significantly affect binary classification performance.

9. Links and References

UCI Dataset:

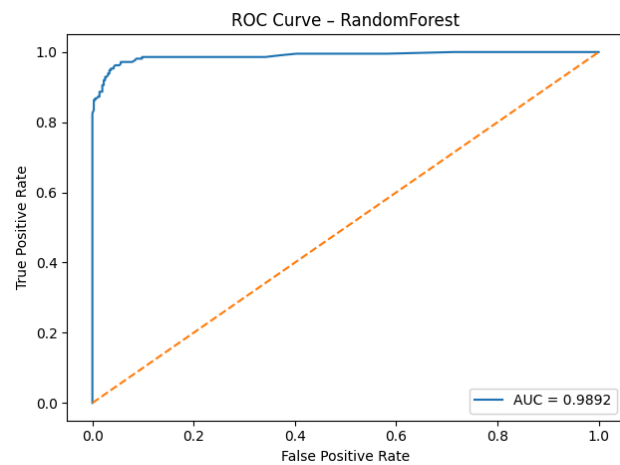
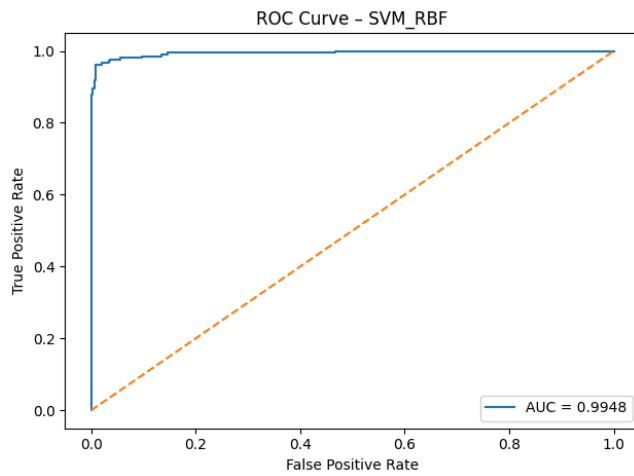
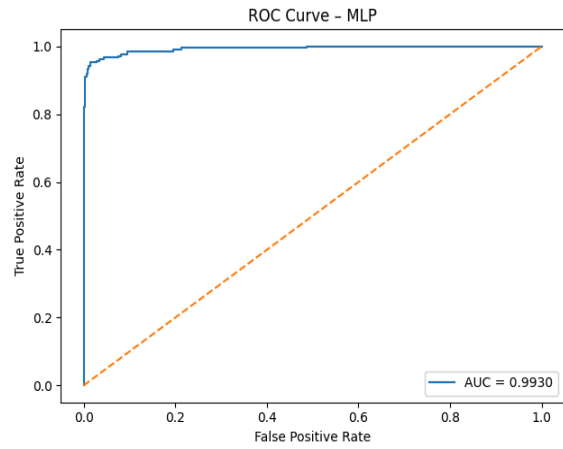
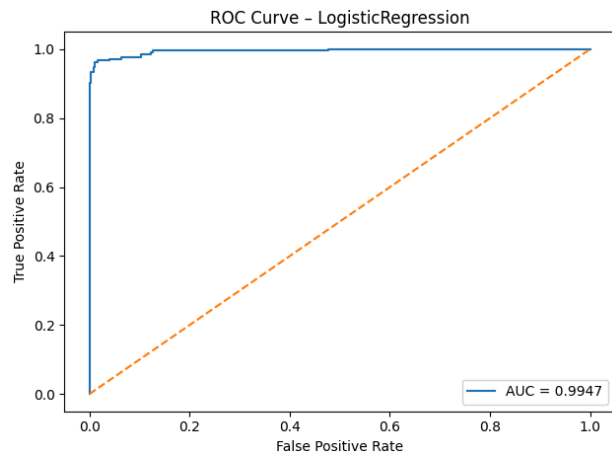
<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Scikit-learn Documentation: <https://scikit-learn.org>

ROC/AUC Tutorial by Fawcett

Github Link: https://github.com/tw237njit/walters_taymar_finalproj

10. Screenshots



Logistic regression 10 fold metrics table

TP	TN	FP	FN	P	N	TPR	TNR	FPR	FNR	Accuracy	BalancedAccuracy	Precision	Recall	F1	ErrorRate	TSS	HSS	BS	BSS	Fold	
21	35	0	1	22	35	0.954545	1	0	0.045455	0.982456		0.977272727	1	0.954545	0.976744	0.017544	0.954545	0.962672	0.027438	0.884226	1
22	34	1	0	22	35	1	0.971429	0.028571	0	0.982456		0.985714286	0.956521739	1	0.977778	0.017544	0.971429	0.963297	0.011566	0.951198	2
20	34	2	1	21	36	0.952381	0.944444	0.055556	0.047619	0.947368		0.948412698	0.909090909	0.952381	0.930233	0.052632	0.896825	0.888016	0.022887	0.901642	3
18	36	0	3	21	36	0.857143	1	0	0.142857	0.947368		0.928571429	1	0.857143	0.923077	0.052632	0.857143	0.883436	0.034913	0.849956	4
20	36	0	1	21	36	0.952381	1	0	0.047619	0.982456		0.976190476	1	0.952381	0.97561	0.017544	0.952381	0.961924	0.02436	0.89531	5
18	36	0	3	21	36	0.857143	1	0	0.142857	0.947368		0.928571429	1	0.857143	0.923077	0.052632	0.857143	0.883436	0.042543	0.817166	6
21	36	0	0	21	36	1	1	0	0	1		1	1	1	1	0	1	1	0.004503	0.980647	7
21	35	1	0	21	36	1	0.972222	0.027778	0	0.982456		0.986111111	0.954545455	1	0.976744	0.017544	0.972222	0.962672	0.017773	0.923618	8
20	36	0	1	21	36	0.952381	1	0	0.047619	0.982456		0.976190476	1	0.952381	0.97561	0.017544	0.952381	0.961924	0.018045	0.922451	9
21	35	0	0	21	35	1	1	0	0	1		1	1	1	1	0	1	1	0.003176	0.98645	10
20.2	35.3	0.4	1	21.2	35.7	0.952597	0.98881	0.01119	0.047403	0.975439		0.970703463	0.98201581	0.952597	0.965887	0.024561	0.941407	0.946738	0.02072	0.911266	Average

MLP 10 fold metrics table

TP	TN	FP	FN	P	N	TPR	TNR	FPR	FNR	Accuracy	BalancedPrecision	Precision	Recall	F1	ErrorRate	TSS	HSS	BS	BSS	Fold
22	33	2	0	22	35	1	0.942857	0.057143	0	0.964912	0.971429	0.916667	1	0.956522	0.035088	0.942857	0.927203	0.020931	0.911683	1
22	34	1	0	22	35	1	0.971429	0.028571	0	0.982456	0.985714	0.956522	1	0.977778	0.017544	0.971429	0.963297	0.005042	0.978724	2
20	34	2	1	21	36	0.952381	0.944444	0.055556	0.047619	0.947368	0.948413	0.909091	0.952381	0.930233	0.052632	0.896825	0.888016	0.051227	0.779847	3
19	36	0	2	21	36	0.904762	1	0	0.095238	0.964912	0.952381	1	0.904762	0.95	0.035088	0.904762	0.923077	0.035439	0.847698	4
20	35	1	1	21	36	0.952381	0.972222	0.027778	0.047619	0.964912	0.962302	0.952381	0.952381	0.952381	0.035088	0.924603	0.924603	0.031709	0.863728	5
17	36	0	4	21	36	0.809524	1	0	0.190476	0.929825	0.904762	1	0.809524	0.894737	0.070175	0.809524	0.842975	0.061909	0.733937	6
21	36	0	0	21	36	1	1	0	0	1	1	1	1	1	0	1	1	1.51E-06	0.999994	7
21	34	2	0	21	36	1	0.944444	0.055556	0	0.964912	0.972222	0.913043	1	0.954545	0.035088	0.944444	0.92607	0.030092	0.870677	8
20	36	0	1	21	36	0.952381	1	0	0.047619	0.982456	0.97619	1	0.952381	0.97561	0.017544	0.952381	0.961924	0.017307	0.925622	9
20	35	0	1	21	35	0.952381	1	0	0.047619	0.982143	0.97619	1	0.952381	0.97561	0.017857	0.952381	0.961538	0.008459	0.963908	10
20.2	34.9	0.8	1	21.2	35.7	0.952381	0.97754	0.02246	0.047619	0.96839	0.96496	0.96477	0.952381	0.956741	0.03161	0.929921	0.93187	0.026212	0.887582	Average

Random Forrest 10 fold metrics table

TP	TN	FP	FN	P	N	TPR	TNR	FPR	FNR	Accuracy	BalancedPrecision	Recall	F1	ErrorRate	TSS	HSS	BS	BSS	Fold	
21	33	2	1	22	35	0.954545	0.942857	0.057143	0.045455	0.947368	0.948701	0.913043	0.954545	0.933333	0.052632	0.897403	0.889891	0.03273	0.861895	1
22	34	1	0	22	35	1	0.971429	0.028571	0	0.982456	0.985714	0.956522	1	0.977778	0.017544	0.971429	0.963297	0.020905	0.91179	2
20	36	0	1	21	36	0.952381	1	0	0.047619	0.982456	0.97619	1	0.952381	0.97561	0.017544	0.952381	0.961924	0.024318	0.895491	3
16	35	1	5	21	36	0.761905	0.972222	0.027778	0.238095	0.894737	0.867063	0.941176	0.761905	0.842105	0.105263	0.734127	0.764463	0.067114	0.711571	4
19	35	1	2	21	36	0.904762	0.972222	0.027778	0.095238	0.947368	0.938492	0.95	0.904762	0.926829	0.052632	0.876984	0.885772	0.039374	0.830784	5
20	36	0	1	21	36	0.952381	1	0	0.047619	0.982456	0.97619	1	0.952381	0.97561	0.017544	0.952381	0.961924	0.032494	0.860353	6
20	35	1	1	21	36	0.952381	0.972222	0.027778	0.047619	0.964912	0.962302	0.952381	0.952381	0.952381	0.035088	0.924603	0.924603	0.026418	0.886466	7
21	32	4	0	21	36	1	0.888889	0.111111	0	0.929825	0.944444	0.84	1	0.913043	0.070175	0.888889	0.854962	0.033211	0.85727	8
18	35	1	3	21	36	0.857143	0.972222	0.027778	0.142857	0.929825	0.914683	0.947368	0.857143	0.9	0.070175	0.829365	0.846154	0.043233	0.814202	9
21	35	0	0	21	35	1	1	0	0	0	1	1	1	1	0	1	1	0.008886	0.962086	10
19.8	34.6	1.1	1.4	21.2	35.7	0.93355	0.969206	0.030794	0.06645	0.95614	0.951378	0.950049	0.93355	0.939669	0.04386	0.902756	0.905299	0.032868	0.859191	Average

SVM RBF 10 fold metrics table

TP	TN	FP	FN	P	N	TPR	TNR	FPR	FNR	Accuracy	BalancedPrecision	Recall	F1	ErrorRate	TSS	HSS	BS	BSS	Fold	
22	35	0	0	22	35	1	1	0	0	1	1	1	1	0	1	1	0.009168	0.961316	1	
21	35	0	1	22	35	0.954545	1	0	0.045455	0.982456	0.977273	1	0.954545	0.976744	0.017544	0.954545	0.962672	0.0113	0.952319	2
20	34	2	1	21	36	0.952381	0.944444	0.055556	0.047619	0.947368	0.948413	0.909091	0.952381	0.930233	0.052632	0.896825	0.888016	0.041406	0.822054	3
19	36	0	2	21	36	0.904762	1	0	0.095238	0.964912	0.952381	1	0.904762	0.95	0.035088	0.904762	0.923077	0.033056	0.857938	4
20	35	1	1	21	36	0.952381	0.972222	0.027778	0.047619	0.964912	0.962302	0.952381	0.952381	0.952381	0.035088	0.924603	0.924603	0.026884	0.884461	5
19	36	0	2	21	36	0.904762	1	0	0.095238	0.964912	0.952381	1	0.904762	0.95	0.035088	0.904762	0.923077	0.027293	0.882704	6
21	36	0	0	21	36	1	1	0	0	1	1	1	1	1	0	1	0.005513	0.976307	7	
21	33	3	0	21	36	1	0.916667	0.083333	0	0.947368	0.958333	0.875	1	0.933333	0.052632	0.916667	0.890173	0.034848	0.850237	8
20	36	0	1	21	36	0.952381	1	0	0.047619	0.982456	0.97619	1	0.952381	0.97561	0.017544	0.952381	0.961924	0.021022	0.909654	9
21	35	0	0	21	35	1	1	0	0	1	1	1	1	1	0	1	1	0.002449	0.989549	10
20.4	35.1	0.6	0.8	21.2	35.7	0.962121	0.983333	0.016667	0.037879	0.975439	0.972727	0.973647	0.962121	0.96683	0.024561	0.945455	0.947354	0.021294	0.908654	Average

Models Summary averages

TP	TN	FP	FN	P	N	TPR	TNR	FPR	FNR	Accuracy	BalancedPrecision	Recall	F1	ErrorRate	TSS	HSS	BS	BSS	Fold	Model	
20.2	35.3	0.4	1	21.2	35.7	0.952597	0.98881	0.01119	0.047403	0.975439	0.970703	0.982016	0.952597	0.965887	0.024561	0.941407	0.946738	0.02072	0.911266	Average	LogisticRegression
20.4	35.1	0.6	0.8	21.2	35.7	0.962121	0.983333	0.016667	0.037879	0.975439	0.972727	0.973647	0.962121	0.96683	0.024561	0.945455	0.947354	0.021294	0.908654	Average	SVM_RBF
19.8	34.6	1.1	1.4	21.2	35.7	0.93355	0.969206	0.030794	0.06645	0.95614	0.951378	0.950049	0.93355	0.939669	0.04386	0.902756	0.905299	0.032868	0.859191	Average	RandomForest
20.2	34.9	0.8	1	21.2	35.7	0.952381	0.97754	0.02246	0.047619	0.96839	0.96496	0.96477	0.952381	0.956741	0.03161	0.929921	0.93187	0.026212	0.887582	Average	MLP